PUC-Rio – Departamento de Informática
Ciência da Computação
Introdução à Arquitetura de Computadores
Prof.: Alexandre Meslin

Trabalho 1 – 2026.1

Instruções Gerais:

Leia com atenção o enunciado do trabalho e as instruções para a entrega. Em caso de dúvidas, não invente. Pergunte!

- O trabalho deve ser entregue até a data marcada no EaD.
- Trabalhos entregues com atraso perderão um ponto por dia (ou fração) de atraso.
- Trabalhos que não compilem (isto é, que não produzam um executável) não serão considerados! Ou seja, receberão grau zero.
- Os trabalhos podem ser feitos em grupos de no máximo dois alunos.
- Alguns grupos poderão ser chamados para apresentações orais ou demonstrações dos trabalhos entregues.
- Importante: Somente ***UM*** integrante do grupo deve fazer a carga dos arquivos fontes no EAD.

Objetivo:

O objetivo deste trabalho é fazer a primeira otimização na operação de soluções de equações lineares a partir da implementação da versão escalar e comparar o resultado com a versão desenvolvida em sala de aula (versão escalar) para calcular o *speedup*. Nenhuma mudança deve ser realizada nos módulos *equation_test.cpp*, *comum.cpp* e no arquivo cabeçalho *comum.h* do programa base já implementado e testado. Apenas a função *processaVetores* do módulo *vetorial_lib.cpp* deve sofrer alterações.

O programa de teste obtém o tempo antes da chamada de uma função da biblioteca e depois do retorno da função para calcular os tempos parciais da execução de cada função. Todas as tomadas de tempo e impressão das medidas de tempo devem ser realizadas na função `main` do programa de teste.

Parte I:

Aprimorar o módulo escrito em linguagem C, chamado *escalar_lib.cpp*, implementado em sala de aula, com a utilização de **instruções vetoriais (AVX/FMA)** usando a biblioteca **Intel Intrinsics**. O novo módulo deverá se chamar *vetorial_lib.cpp*.

Considere um sistema de equações do tipo:

$$a_{0,0}x_0 + a_{0,1}x_1 + a_{0,2}x_2 + \ldots + a_{0,n-1}x_{n-1} = b_0$$

$$a_{1,0}x_0 + a_{1,1}x_1 + a_{1,2}x_2 + \ldots + a_{1,n-1}x_{n-1} = b_1$$

$$a_{2,0}x_0 + a_{2,1}x_1 + a_{2,2}x_2 + \ldots + a_{2,n-1}x_{n-1} = b_2$$

$$\ldots$$

$$a_{n-1,0}x_0 + a_{n-1,1}x_1 + a_{n-1,2}x_2 + \ldots + a_{n-1,n-1}x_{n-1} = b_{n-1}$$

Este sistema pode ser também descrito de forma matricial. Seja $A \in \mathbb{R}^{n \times n}$, $x \in \mathbb{R}^n$ e $b \in \mathbb{R}^n$, então:

$$Ax = b$$

Onde a matriz *A* representa os coeficientes da equação que multiplicam o vetor *x* como mostrado a seguir:

$$A = \begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & \ldots & a_{0,n-1} \\ a_{1,0} & a_{1,1} & a_{1,2} & \ldots & a_{1,n-1} \\ a_{2,0} & a_{2,1} & a_{2,2} & \ldots & a_{2,n-1} \\ & & \ldots & & \\ a_{n-1,0} & a_{n-1,1} & a_{n-1,2} & \ldots & a_{n-1,n-1} \end{bmatrix}$$

Os vetores *x* e *b* podem ser escritos da seguinte forma:

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ x_{n-1} \end{bmatrix}$$

$$b = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \dots \\ b_{n-1} \end{bmatrix}$$

Considerando que as *n* equações são linearmente independentes, o sistema possui apenas uma resposta para os valores do vetor *x* e operações de soma entre linhas e multiplicações dentro de uma linha não modificam a resposta do sistema. Por exemplo, podemos multiplicar a linha *i* por um fator *k* e usar essa nova equação no lugar da equação da linha *i*. Outra operação permitida é somar a linha *i* com a linha *j* e substituir a linha *i* (ou a linha *j*) pelo resultado da operação.

Implementar a função `processaVetores` que transforma a matriz *A* e o vetor *b*. Essa função deverá zerar todos os valores abaixo da diagonal principal da matriz *A*, mantendo integro o sistema de equações. Essa função deverá utilizar instruções vetoriais (**AVX/FMA**). O uso dessas instruções será o foco desse trabalho.

A função deverá receber um ponteiro para a matriz *A* e outro para o vetor *b* assim como um inteiro representando o número de incógnitas do sistema de acordo com o protótipo a seguir.

```
void processaVetores(double *mA, double *vB, int nIncognitas);
```

A função deverá modificar a matriz *A* e o vetor *b* para zerar toda a matriz *A* abaixo da diagonal principal.

Parte II:

Use o programa em linguagem C, chamado *equation*, que implementa um código para testar a biblioteca *vetorial_lib.cpp*. Esse programa deve receber:

a) o nome do arquivo binário com a matriz *A* (-m <arquivo>);
b) o nome do arquivo binário com o vetor *b* (-v <arquivo>);
c) a quantidade de incógnitas (-n <quantidade de incógnitas>
d) opcionalmente, o nome do arquivo de saída para contabilizar o tempo (-o <arquivo>).

Os arquivos *equation_test.cpp* e *comum.cpp* implementam essas funcionalidades e realizam a chamada correta para a função *processaVetores*. Esses arquivos utilizam os *headers comum.h* e *gpu.h* (ainda sem utilidade nessa versão).

Exemplo de linha de comando:

```
equation -m matrizA.bin -n 32 -o tempos.txt -v vetorB.bin
```

Onde:

- `32` é o número de incógnitas do sistema;
- `matrizA.bin` é o nome do arquivo com 32x32 *floats* representando a matriz *A*;
- `vetorB.bin` é o nome do arquivo com 32 *floats* representando o vetor *b*.

A função principal cronometra o tempo de execução da função *processaVetores*. Para marcar o início e o final do tempo em cada uma das situações, usamos a função padrão `clock` prototipada em `<time.h>`. Essa função retorna o tempo aproximado de processador usado pelo programa. Para calcular a diferença de tempo (delta) entre duas marcas de tempo *t0* e *t1*, usamos a macro `timedifference_msec`, definida no módulo `comum.h`, fornecido.

Entregável:

Você deve entregar um relatório informado o que funcionou e o que não funcionou no seu programa. Esse relatório deve conter também um ou mais gráficos comparando a versão implementada em sala de aula com a sua versão usando a biblioteca **Intel Intrincs**.

Realize testes com diversas quantidades de incógnitas, desde quantidades muito pequenas até quantidades realmente enormes, do tamanho máximo que a máquina de testes permitir. Mas sempre com quantidades múltiplas de oito.

O programa pode inicialmente ser testado com sistemas pequenos para facilitar a depuração, mas a versão final deve ser testada com sistemas grandes, com dimensão 1024 x 1024 ou superior.

Além do relatório, você deve entregar também o arquivo fonte *vetorial_lib.cpp*. Lembre-se que você não modificou os arquivos *comum.cpp*, *comum.h* e *equation_test.cpp*, logo, esses arquivos não precisam e nem devem ser entregues.

Observação 1:

Todas as práticas da disciplina serão executadas no ambiente Linux. Como o objetivo é usar esse trabalho como base para os próximos trabalhos desta disciplina, este trabalho deve ser compilado e executado no ambiente Linux. Para fazer a compilação do programa no Linux, deve-se usar o GCC, com os seguintes argumentos:

```
$ gcc -Wextra -Wall -mavx -march=native -o equation equation_test.cpp comum.cpp
vetorial_lib.cpp
```

Onde,

- `equation` = nome do programa executável.
- `equation_test.cpp` = nome do programa fonte que tem a função `main()`.
- `vetorial_lib.cpp` = nome do modulo fonte da biblioteca que você criou.
- `comum.cpp` = nome do módulo fonte que contém as funções auxiliares comuns a todas as tarefas.

Não há motivos para haver mensagens de advertências ou erros por parte do compilador ou ligador.

Uma máquina virtual padrão **VMware** com o sistema **Linux** está disponível na área de download do site da **Equipe de Suporte do DI**, em:

URL: `http://suporte.inf.puc-rio.br/download/vms/VMCCPP-FC23-64-DI-PUC-Rio-V1.0.zip`

Para executar a máquina virtual, basta baixar o **VMware Workstation Player** gratuitamente a partir do site da **VMware**.

URL: `https://www.vmware.com/br/products/workstation-player/workstation-player-evaluation.html`

O servidor do DI está disponível para acesso remoto, conforme informado anteriormente, e pode ser usado para executar o programa de teste.

Observação 2:

Apenas o programa fonte *vetorial_lib.cpp* deve ser carregado no site de EaD da disciplina até o prazo de entrega.

Observação 3:

Crie e documente no seu relatório os códigos de erro que você usou na sua biblioteca.

Observação 4:

Documente o seu código no estilo Javadoc (Doxygen).

Bom trabalho